

**PROYEK AKHIR
UJIAN AKHIR SEMESTER
METODE PERAMALAN**



Yogyakarta, 22 Juni 2024

Oleh:

1. Yustian Faza (22/497588/PA/21430)
2. Cintya Kusumawardhani (22/502898/PA/21591)
3. Fabima Fadhlin Minallah Zidta (22/504936/PA/21729)

Dosen Pengampu : 1. Dr. Drs. Gunardi, M.Si.

2. Mohamad Fahruli Wahyujati, S.Si., M.Si.

**PROGRAM STUDI STATISTIKA
DEPARTEMEN MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA**

2024

Optimasi Program Gizi Prabowo-Gibran: Prediksi Produksi Susu Indonesia sebagai Fondasi Indonesia Emas 2045

Cintya Kusumawardhani¹, Fabima Fadhlin Minallah Zidta², Yustian Faza³

¹ Yogyakarta, Universitas Gadjah Mada

² Yogyakarta, Universitas Gadjah Mada

³ Yogyakarta, Universitas Gadjah Mada

Abstrak

Sapi dan susu sapi merupakan komponen utama dalam keberlangsungan program unggulan calon presiden terpilih periode 2024-2029, Prabowo Subianto, yaitu program Makan Siang dan Susu Gratis untuk anak-anak di seluruh Indonesia. Untuk memastikan bahwa susu sapi yang akan diproduksi selama program berlangsung dapat menjangkau masyarakat melalui strategi yang efektif, sebuah ekstrapolasi produksi susu Indonesia di masa depan dengan menggunakan *deep forecasting* telah dicoba dalam penelitian ini. Penelitian ini bertujuan untuk mengevaluasi potensi tiga model *deep forecasting* untuk memprediksikan jumlah sapi dan juga produksi susu tahunan Indonesia. Data diperoleh dari *website* FAO. Hasil dari analisis menunjukkan bahwa model yang paling cocok digunakan adalah *Convolutional Neural Network* (CNN) dengan nilai MSE sebesar 0.00387 untuk data jumlah sapi dan 0.00506 untuk data jumlah produksi susu sapi. Prediksi menunjukkan produksi susu di Indonesia pada tahun 2029 diperkirakan mencapai 1.014.361,56 ton, tetapi angka ini mengalami penurunan dari tahun sebelumnya. Di sisi lain, jumlah sapi di Indonesia diperkirakan akan naik mencapai 18.055.802 ekor. Namun secara matematis, kedua komponen ini masih belum cukup untuk memenuhi target susu gratis untuk seluruh anak-anak di Indonesia. Pemerintah masih tetap perlu melakukan impor susu sapi maupun sapi. Pemerintah juga perlu menyiapkan langkah strategis untuk memastikan ketersediaan susu yang cukup sambil meminimalkan ketergantungan pada impor jangka panjang.

Keywords: Susu Gratis, *Deep Learning*, *Convolutional Neural Network*, LSTM, Bi-LSTM

1 PENDAHULUAN

Indonesia saat ini menghadapi tantangan besar dalam memenuhi kebutuhan gizi masyarakat, terutama bagi anak-anak dan ibu hamil, yang merupakan kelompok rentan terhadap masalah kekurangan gizi dan stunting. Stunting atau tengkes merupakan kondisi yang mengindikasikan kekurangan gizi kronis yang mengakibatkan terganggunya pertumbuhan fisik dan kognitif anak (WHO, 2015). Masalah ini perlu segera ditangani secara langsung dan massal oleh pemerintah untuk memastikan

tercapainya kualitas Sumber Daya Manusia (SDM) dan kualitas hidup yang baik.

Hasil riset Center for Indonesian Policy Studies (CIPS), sekitar 21 juta orang, atau sekitar 7% dari populasi Indonesia, mengalami kekurangan gizi dengan asupan kalori per kapita harian di bawah standar Kementerian Kesehatan yang sebesar 2.100 kkal. Selain itu, pada tahun lalu, 21,6% anak Indonesia di bawah usia lima tahun mengalami stunting, yang mengindikasikan rasio tinggi berbanding usia yang rendah. Tidak

hanya itu, sekitar 7,7% anak lainnya menderita wasting, yang merupakan kondisi rendahnya rasio berat badan berbanding tinggi badan.

Pemerintahan Prabowo-Gibran telah mencanangkan program ambisius untuk mengatasi masalah ini melalui pemberian makan siang dan susu gratis di sekolah-sekolah serta bantuan gizi bagi ibu hamil dan balita di seluruh Indonesia. Program ini ditargetkan akan menjangkau lebih dari 80 juta penerima manfaat dengan cakupan 100% pada tahun 2029. Inisiatif ini diharapkan dapat meningkatkan kesehatan anak-anak dan ibu hamil, serta membantu ekonomi keluarga.

Namun, di tengah upaya ini, muncul persoalan baru terkait defisit neraca susu nasional. Program Minum Susu Gratis diproyeksikan akan meningkatkan kebutuhan susu tahunan dari 4,6 juta ton menjadi 8,7 juta ton pada tahun 2029, sementara produksi susu tahunan Indonesia saat ini hanya sekitar 0,9 juta ton. Hal ini mengakibatkan defisit sebesar 7,8 juta ton per tahun, yang setara dengan kebutuhan impor sekitar 2 juta ekor sapi perah.

Dalam tujuh tahun terakhir, data menunjukkan bahwa kebutuhan susu

nasional meningkat rata-rata 6% per tahun, sedangkan produksi susu hanya meningkat 1% per tahun. Ketidakseimbangan ini mengharuskan Indonesia mengimpor susu setiap tahun untuk memenuhi kebutuhan nasional. Dalam mendukung program ini, pemerintah merencanakan impor sapi perah sebanyak 1,1 juta ekor dengan anggaran sekitar Rp 90 triliun. Selain itu, data dari Badan Pusat Statistik (BPS) mencatat bahwa sepanjang tahun 2023, Indonesia mengimpor susu senilai US\$ 921.425.519 atau setara dengan Rp 14,324 triliun, dengan volume impor mencapai 287 ribu ton.

Mengingat tantangan yang ada, sangat penting untuk mengevaluasi dan mengoptimalkan strategi yang diterapkan oleh pemerintahan Prabowo-Gibran terkait dengan Program Gizi. Fokus penelitian ini adalah pada evaluasi dan optimalisasi produksi susu di Indonesia menggunakan pendekatan statistik, khususnya dengan menggunakan metode peramalan *deep forecasting* atau *neural network forecasting*. Tujuan jangka panjangnya adalah mencapai Indonesia Emas 2045 dengan memberikan rekomendasi komprehensif untuk meningkatkan produksi susu nasional, mengurangi

ketergantungan pada impor, dan memastikan keberlanjutan program gizi yang diluncurkan oleh pemerintah. Optimalisasi program ini akan menjadi fondasi penting bagi pencapaian visi Indonesia Emas 2045, di mana kualitas SDM yang unggul dan sehat menjadi faktor kunci dalam pembangunan bangsa.

2 DATA

2.1 Penjelasan Data

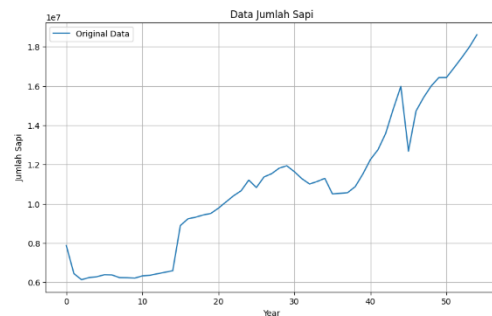
Data yang digunakan dalam proyek penelitian ini terdiri dari data jumlah sapi nasional dan data jumlah produksi susu sapi nasional dengan periode tahunan. Data ini diperoleh dari *website* Food and Agriculture Organization (FAO).

Kedua data ini akan digunakan untuk analisis *forecasting* ke depannya. Kedua data akan dianalisis masing-masing untuk mengetahui nilainya beberapa tahun yang akan datang.

Variabel	Periode
Jumlah Sapi	1968-2020
Produksi Susu	1968-2020

2.2 Eksplorasi Data

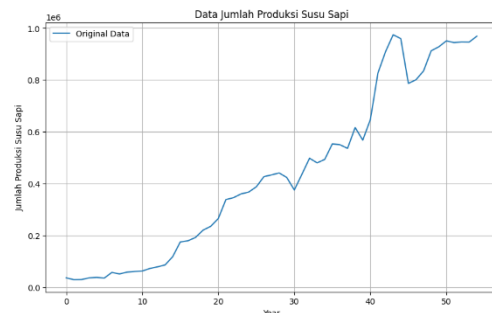
2.2.1 Data Jumlah Sapi



Gambar 1. Plot Data Jumlah Sapi

Dari plot di atas dapat terlihat bahwa jumlah sapi di Indonesia terus meningkat dari tahun ke tahun. Hal ini memberikan harapan bagi terlaksananya program Susu Gratis.

2.2.2 Data Jumlah Produksi Susu



Gambar 2. Plot Produksi Susu Sapi

Dari plot di atas juga dapat terlihat bahwa jumlah produksi susu sapi di Indonesia terus meningkat. Akan tetapi, terdapat sedikit penurunan di beberapa tahun terakhir. Hal ini dapat menjadi perhatian khusus bagi pemerintah.

2.3 Pra-Pemrosesan Data

2.3.1 Penskalaan Data

Penskalaan data adalah proses mengubah nilai fitur-fitur dataset hingga berada dalam rentang tertentu, misalnya 0 hingga 1. Hal ini dilakukan untuk memastikan tidak ada satu fitur pun yang mendominasi penghitungan jarak dalam sebuah algoritma (Wei, 2023).

Metode penskalaan yang digunakan dalam penelitian ini adalah *Min-Max Scaling*. Metode ini dipilih karena dapat mempercepat proses *training* dan membantu model mencapai konvergensi lebih cepat. Formula dasarnya adalah sebagai berikut:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

3 METODE PENELITIAN

3.1 Metode Deep Forecasting

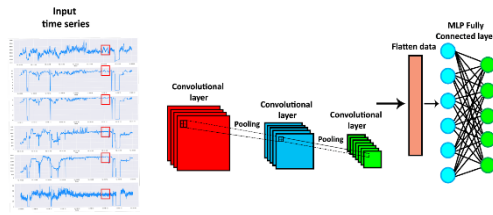
Deep forecasting adalah pendekatan dalam analisis dan prediksi data deret waktu yang menggunakan teknik-teknik deep learning. Metode ini difokuskan pada kemampuan untuk mengekstraksi pola-pola kompleks dan sinyal-sinyal lemah dari data deret waktu yang besar dan kompleks (Benidis et al., 2022).

Teknik *deep forecasting* sering kali melibatkan penggunaan lapisan-lapisan yang dalam (*deep layers*) dalam *neural network* (Zarai et al., 2020), yang memungkinkan model untuk memahami hubungan yang kompleks antara berbagai faktor dalam data deret waktu. Model *deep forecasting* juga dapat mengatasi tantangan dalam analisis deret waktu yang melibatkan banyak variabel terkait dan dapat menghadapi data dengan catatan pendek, meskipun pengumpulan data yang memadai sering kali sulit di aplikasi dunia nyata (Benidis et al., 2022).

3.2 Convolutional Neural Network (CNN)

Menurut Casolaro et al. (2023) *Convolutional Neural Network* atau CNN merupakan salah satu metode *deep forecasting* yang digunakan untuk meramalkan peristiwa jangka pendek dengan memanfaatkan lapisan konvolusi, pooling, dan *fully connected*, seperti pada Gambar 2. CNN memiliki tiga fitur kunci, yaitu koneksi lokal, pembagian bobot, dan equivariance translasi. Koneksi lokal mengacu pada interaksi setiap neuron CNN dengan area masukan eksklusif yang disebut sebagai *receptive field*. Selain itu,

neuron dalam satu lapisan menggunakan matriks bobot yang identik. *Equivariance* translasi merujuk pada kemampuan CNN untuk mengenali pola tertentu tanpa memperhatikan letak pasti pola tersebut dalam masukan.



Gambar 3. Algoritma CNN (Casolaro et al., 2023)

Dalam konteks peramalan deret waktu, konvolusi 1D digunakan untuk mengekstrak pola dan informasi penting dari data deret waktu yang dapat digunakan untuk memprediksi nilai-nilai di masa depan. Secara formal, jika $\mathcal{X}$ adalah deret waktu input dengan panjang L dan w adalah kernel atau filter dengan panjang q , maka hasil dari konvolusi 1D pada posisi t dalam deret waktu didefinisikan sebagai:

$$y(t) = (w * \mathcal{X})(t) = \sum_{a=-q/2}^{q/2} w(a) \mathcal{X}(t-a) \quad \forall t \in [1, \dots, L]$$

Di sini, t mewakili posisi dalam deret waktu $\mathcal{X}$ di mana hasil konvolusi $y(t)$ dihitung. Operasi ini berulang untuk setiap titik waktu dalam deret waktu, memungkinkan pengambilan

fitur-fitur yang relevan dari data yang berjalan.

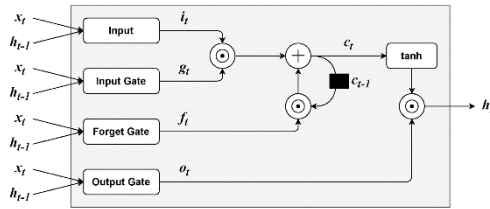
Kelemahan utama CNN untuk peramalan deret waktu adalah penggunaan kernel Euclidean, yang hanya mempertimbangkan subsekuen waktu berkelanjutan dan umumnya pendek pada satu waktu. Namun, dalam tugas peramalan deret waktu, dapat berguna untuk mengekstrak fitur representatif dengan menganalisis sampel-sampel deret waktu yang tidak berurutan dan lebih jauh di masa lalu.

3.3 Metode Long-Short

Term Memory (LSTM)

Long Short-Term Memory atau LSTM merupakan salah satu metode deep forecasting yang efektif dalam mengatasi masalah hilangnya gradien. LSTM mampu mempertahankan dan mengingat informasi baik jangka pendek maupun jangka panjang sehingga sesuai untuk menyelesaikan permasalahan non-linear (Cesarini et al., 2024). Struktur LSTM terdiri dari dua komponen utama, yaitu state dan gate. Komponen state melibatkan hidden state, yang bertanggung jawab atas penyimpanan nilai dari state tersembunyi sebelumnya dan berperan sebagai memori dari RNN, serta cell state, yang merupakan hasil gabungan

dari data masukan dan state tersembunyi sebelumnya. Sementara itu, terdapat tiga jenis gate dalam struktur LSTM (Casolaro et al., 2023), yaitu *forget gate* yang menentukan apakah informasi dari state saat ini layak dipertahankan, *input gate* yang menentukan apakah state internal harus dijadikan sebagai memori sel, dan *output gate* yang menentukan isi dari state tersembunyi berikutnya.



Gambar 3. Algoritma LSTM

3.4 Metode *Bidirectional Long Short-Term Memory (Bi-LSTM)*

Menurut Staudemeyer dan Morris (2019), *Bidirectional Long Short-Term Memory* atau Bi-LSTM adalah varian dari jaringan saraf *Recurrent Neural Network* (RNN) yang memungkinkan analisis tidak hanya terhadap data masa lalu dari suatu titik dalam urutan (*sequence*), tetapi juga terhadap data masa depan dari titik tersebut. Pendekatan ini memanfaatkan dua jaringan LSTM terpisah yang masing-masing menerima urutan input dalam arah maju dan mundur. Kemudian,

hasilnya digabungkan ke lapisan output yang sama.

Dengan menggunakan *Bidirectional LSTM*, model dapat menghilangkan batasan analisis hanya pada satu arah (biasanya masa lalu) seperti pada LSTM konvensional sehingga memungkinkan untuk menangkap pola dan hubungan yang lebih kompleks dalam data urutan, terutama dalam konteks prediksi di mana informasi dari masa depan juga penting untuk dipertimbangkan.

3.5 Matrik Evaluasi

MSE (*Mean Squared Error*) adalah salah satu metrik evaluasi yang umum digunakan dalam pemodelan statistik dan machine learning untuk mengukur seberapa dekat prediksi model dengan nilai sebenarnya dari data yang diuji. Secara matematis, MSE didefinisikan sebagai rata-rata dari kuadrat selisih antara nilai prediksi (*predicted values*) dan nilai sebenarnya (*actual values*) dari data.

Secara formal, MSE untuk sebuah set data dengan n observasi dihitung sebagai berikut:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Intuisi di balik MSE adalah bahwa semakin kecil nilai MSE, semakin baik model dalam melakukan

prediksi yang akurat. Nilai MSE yang lebih kecil menunjukkan bahwa perbedaan antara nilai prediksi dan nilai sebenarnya cenderung lebih kecil secara rata-rata.

4 HASIL DAN PEMBAHASAN

Dalam penelitian ini, dilakukan perbandingan antara tiga model untuk

forecasting: CNN (*Convolutional Neural Network*), LSTM (Long Short-Term Memory), dan Bi-LSTM (*Bidirectional LSTM*). Ketiga model ini dievaluasi menggunakan *cross-validation*, dengan nilai mean *loss* atau mean MSE (Mean Squared Error) sebagai metrik evaluasi utama.

Table 2. Hasil Evaluasi Model

Model	MSE Data Sapi	MSE Data Susu Sapi
CNN	0.00387853787	0.00506787458
LSTM	0.00558516478	0.00528496461
Bi-LSTM	0.00492734524	0.00512189170

4.1 Arsitektur CNN

- **Convolutional Layer:** Terdiri dari 64 filter dengan kernel size 2 dan fungsi aktivasi ReLU untuk mengekstraksi fitur dari input data. Input shape dari layer ini disesuaikan dengan dimensi data yang diinputkan.
- **MaxPooling Layer:** Digunakan untuk mereduksi dimensi hasil dari convolutional layer dengan ukuran pool_size=2.
- **Flatten Layer:** Mengubah output dari MaxPooling layer menjadi vektor satu dimensi agar dapat dihubungkan dengan dense layer.

- **Dense Layer:** Terdiri dari 50 unit dengan fungsi aktivasi ReLU untuk memproses fitur-fitur yang telah diekstraksi sebelumnya.
- **Output Layer:** Dense layer dengan 1 unit tanpa fungsi aktivasi, yang menghasilkan prediksi untuk variabel target.
- **Optimizer dan Loss Function:** Menggunakan optimizer Adam dan *loss function* MSE untuk mengukur perbedaan antara nilai prediksi dengan nilai sebenarnya.

4.2 Arsitektur LSTM

- **LSTM Layer:** Memiliki dua layer LSTM. Layer pertama memiliki 48 unit dengan fungsi

aktivasi ReLU. Layer kedua memiliki 24 unit dengan fungsi aktivasi ReLU, untuk memproses *sequences* yang dihasilkan oleh layer pertama.

- **Output Layer:** Dense layer dengan 1 unit tanpa fungsi aktivasi untuk menghasilkan prediksi.
- **Optimizer dan Loss Function:** Menggunakan *optimizer* Adam dan *loss function* MSE untuk mengukur perbedaan antara nilai prediksi dengan nilai sebenarnya.

4.3 Arsitektur Bi-LSTM

- **Bidirectional LSTM Layer:** Terdiri dari satu layer Bidirectional LSTM dengan 48 unit dan fungsi aktivasi ReLU. Model ini menerima input dalam dua arah, yaitu dari awal ke akhir *sequence* dan sebaliknya.
- **Output Layer:** Dense layer dengan 1 unit tanpa fungsi aktivasi untuk menghasilkan prediksi.
- **Optimizer dan Loss Function:** Menggunakan *optimizer* Adam dan *loss function* MSE untuk mengukur perbedaan antara nilai prediksi dengan nilai sebenarnya.

Dalam semua model ini, data dipisahkan menjadi training dan testing set menggunakan KFold cross-validation dengan 5 fold. Setiap model di-train menggunakan data training dan dievaluasi menggunakan data testing, dengan mencatat loss (MSE) pada setiap fold untuk kemudian dihitung mean loss-nya sebagai hasil evaluasi akhir. Metode ini memastikan bahwa performa dari setiap model dievaluasi secara adil dan robust terhadap variasi dalam data.

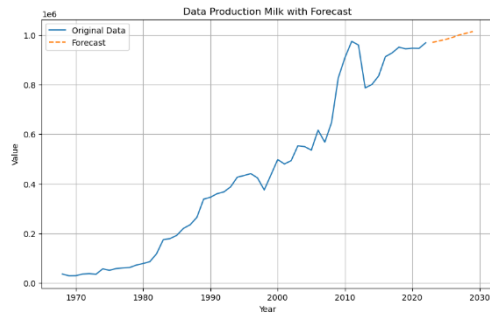
Nilai mean MSE (Mean Squared Error) digunakan untuk mengukur seberapa baik model memprediksi nilai target (dalam kasus ini, produksi Cattle dan Milk) dengan membandingkan nilai prediksi dengan nilai sebenarnya. Berdasarkan hasil yang diberikan:

- **CNN** memiliki mean MSE sebesar 0.0039 untuk data Cattle dan 0.00517 untuk data Milk.
- **LSTM** memiliki mean MSE sebesar 0.0056 untuk data Cattle dan 0.00528 untuk data Milk.
- **Bi-LSTM** memiliki mean MSE sebesar 0.0049 untuk data Cattle dan 0.00512 untuk data Milk.

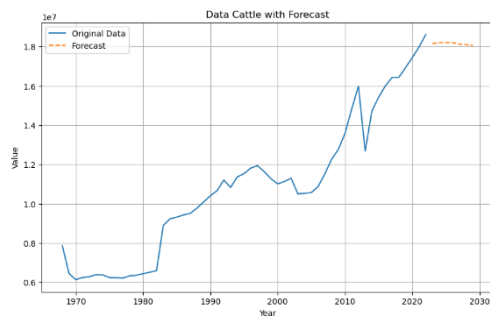
Berdasarkan nilai mean MSE terendah untuk kedua variabel (Cattle

dan Milk), model terbaik yang dapat dipilih adalah CNN.

Diperoleh hasil peramalan dengan menggunakan metode CNN untuk 7 periode ke depan (2022 – 2029) adalah sebagai berikut



Gambar Data Asli dan Peramalan Susu



Gambar Data Asli dan Peramalan Sapi

Berdasarkan hasil peramalan menggunakan metode CNN, produksi susu di Indonesia pada tahun 2029 diprediksi mencapai 1.014.361,56 ton, sementara jumlah sapi diproyeksikan mencapai 18.055.802 ekor. Namun, dengan kebutuhan susu yang diproyeksikan mencapai 8,7 juta ton pada tahun yang sama, terlihat bahwa Indonesia menghadapi defisit besar dalam produksi susu. Secara matematis,

defisit ini berarti bahwa pemerintah perlu mengimpor sekitar 7,686 juta ton susu, dengan estimasi biaya mencapai 384,3 triliun rupiah berdasarkan harga impor susu tahun 2023. Angka ini mencerminkan tantangan signifikan dalam mencapai kemandirian pangan dalam sektor susu, dengan implikasi anggaran yang besar melebihi bahkan anggaran APBN 2024.

Di sisi lain, prediksi menunjukkan bahwa jumlah sapi perah di Indonesia akan terus menurun, dengan hanya sekitar 500 ribu sapi perah produktif dari total 18,6 juta sapi pada tahun 2022. Meskipun diproyeksikan ada sekitar 18,05 juta sapi pada tahun 2029, kebutuhan untuk memenuhi program Susu Gratis masih memerlukan impor tambahan sekitar 2,5 juta ekor sapi perah. Meskipun mengimpor sapi terlihat lebih ekonomis secara nominal, dengan perkiraan anggaran mencapai 205 triliun rupiah untuk mengimpor 2,5 juta ekor sapi, biaya tambahan seperti biaya impor, perawatan, dan produksi dapat membuat total biaya lebih tinggi dan memerlukan evaluasi ekonomi yang lebih mendalam serta pertimbangan strategis.

5 KESIMPULAN DAN SARAN

Dalam penelitian ini, metode Deep Forecasting menggunakan Convolutional Neural Network (CNN) terbukti menjadi pilihan yang paling efektif untuk memprediksi jumlah produksi susu dan jumlah sapi di Indonesia pada tahun 2029. CNN memberikan kinerja prediksi yang lebih baik dibandingkan dengan metode Long Short-Term Memory (LSTM) dan Bidirectional LSTM (Bi-LSTM). CNN berhasil menangkap pola-pola spasial dan temporal yang kompleks dalam data produksi susu dan populasi sapi, yang memberikan hasil prediksi yang lebih akurat dan stabil.

Hasil penelitian yang telah dilakukan, menyoroti kompleksitas dalam memenuhi kebutuhan susu nasional Indonesia, dengan pilihan antara mengimpor susu atau mengimpor sapi perah memiliki implikasi yang signifikan baik dari segi ekonomi maupun keberlanjutan. Diperlukan pendekatan yang komprehensif untuk memastikan ketersediaan susu yang cukup sambil meminimalkan ketergantungan pada impor jangka

panjang, sejalan dengan visi untuk mencapai Indonesia Emas 2045 melalui peningkatan produksi susu nasional yang berkelanjutan.

Pemerintah dan stakeholder terkait perlu mengambil langkah-langkah strategis, seperti meningkatkan investasi dalam infrastruktur teknologi pertanian untuk mendukung implementasi teknologi canggih, memperkuat kebijakan pendukung seperti insentif untuk teknologi modern dan regulasi pro-investasi, serta memperkuat program pendidikan dan pelatihan bagi peternak. Selain itu, diharapkan penelitian selanjutnya, mampu mengevaluasi dampak perubahan lingkungan terhadap produksi susu, dan mengintegrasikan aspek multi-dimensi utamanya dalam sudut pandang ekonomi terkait ketersediaan pangan untuk memberikan kontribusi yang lebih menyeluruh dalam pemenuhan kebutuhan pangan dan gizi masyarakat.

DAFTAR PUSTAKA

- Casolaro, A., Capone, V., Iannuzzo, G., dan Camastra, F. (2023). Deep Learning for Time Series Forecasting: Advances and Open Problems. *Information* 14(11):598. <https://doi.org/10.3390/info14110598>
- Cesarini, L., Gonçalves, R., Martina, M., Romao, X., Monteleone, B., Pereira, F.L., dan Figueiredo, R. (2024). Comparison of deep learning models for milk production forecasting at national scale. *Computers and Electronics in Agriculture* 221(108933): 1-11. <https://doi.org/10.1016/j.compag.2024.108933>
- Firdaus, A.H. dan Nurhayati, E. (2023) *Future Food Demand in Poor Indonesian Districts*. Policy Paper No. 59. Center for Indonesian Policy Studie (CIPS) <https://repository.cips-indonesia.org/media/publications/563231-future-food-demand-in-poor-indonesian-di-c3d255cb.pdf>
- Food and Agriculture Organization of the United Nations (FAO). (2023). *Production / Crops and livestock products – Metadata*. FAO Global Statistical Yearbook, FAO Regional Statistical Yearbooks. <https://www.fao.org/faostat/en/#data/QCL>
- Staudemeyer, R.C. dan Morris, E.R. (2019). *Understanding LSTM -- a tutorial into Long Short-Term Memory Recurrent Neural Networks*. <https://doi.org/10.48550/arXiv.1909.09586>
- Subianto, P. dan Raka, G. R. (2024) *Visi, Misi, dan Program Calon Presiden dan Wakil Presiden 2024-2029 H. Prabowo Subianto Gibran Rakabuming Raka*
- Nutrition and Food Safety (NFS). (2015). *Global Nutrition Targets 2025*. Policy Brief Series. World Health Organization (WHO)
- Zarai, R., Kachout, M., Hazber, M.A.G., dan Mahdi, M.A. (2020) Recurrent Neural Networks & Deep Neural Networks Based on Intrusion Detection System. *Open Access Library Journal*, 7: e6151. <https://doi.org/10.4236/oalib.1106151>

LAMPIRAN

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from numpy import array, hstack
from keras.models import Sequential
from keras.layers import LSTM, Dense, Conv1D, MaxPooling1D, Flatten
from sklearn.model_selection import cross_val_score, TimeSeriesSplit
from sklearn.model_selection import Kfold
from sklearn.preprocessing import MinMaxScaler

data = pd.read_csv('dataset-neural-network-milk-prediction.csv')
def split_sequence(sequence, n_steps):
    X, y = list(), list()
    for i in range(len(sequence)):
        # find the end of this pattern
        end_ix = i + n_steps
        # check if we are beyond the sequence
        if end_ix > len(sequence)-1:
            break
        # gather input and output parts of the pattern
        seq_x, seq_y = sequence[i:end_ix], sequence[end_ix]
        X.append(seq_x)
        y.append(seq_y)
    return array(X), array(y)

## Load Dataset
df = data
x1 = array(df['Cattle'])
x2 = array(df['Milk'])

x1 = x1.reshape((len(x2), 1))
x2 = x2.reshape((len(x2), 1))

plt.figure(figsize=(10, 6))
plt.plot(x1, label='Original Data')
plt.xlabel('Year')
plt.ylabel('Jumlah Sapi')
plt.title('Data Jumlah Sapi')
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(10, 6))
plt.plot(x2, label='Original Data')
plt.xlabel('Year')
plt.ylabel('Jumlah Produksi Susu Sapi')
plt.title('Data Jumlah Produksi Susu Sapi')
plt.legend()
plt.grid(True)
plt.show()

## CNN
n_steps = 5
n_features = 1
```

```

X1, y1 = split_sequence(x1, n_steps)
X1 = X1.reshape((X1.shape[0], X1.shape[1], n_features))

from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
X1_scaled = scaler.fit_transform(X1.reshape(-1, X1.shape[-1])).reshape(X1.shape)
y1_scaled = scaler.fit_transform(y1.reshape(-1, y1.shape[-1])).reshape(y1.shape)
X1_scaled

from numpy import array, hstack
from keras.models import Sequential

n_splits = 5
kf = KFold(n_splits=n_splits, shuffle=True)
fold_no = 1
loss_per_fold = []
for train_index, test_index in kf.split(X1_scaled):
    X_train, X_test = X1_scaled[train_index], X1_scaled[test_index]
    y_train, y_test = y1_scaled[train_index], y1_scaled[test_index]

    model = Sequential()
    model.add(Conv1D(filters=64, kernel_size=2, activation='relu',
input_shape=(n_steps, X1_scaled.shape[2])))
    model.add(MaxPooling1D(pool_size=2))
    model.add(Flatten())
    model.add(Dense(50, activation='relu'))
    model.add(Dense(1))
    model.compile(optimizer='adam', loss='mse')

    print(f"Training for fold {fold_no}...")
    model.fit(X_train, y_train, epochs=200, verbose=0)

    loss = model.evaluate(X_test, y_test, verbose=0)
    print(f"Fold {fold_no} loss: {loss}")

    loss_per_fold.append(loss)
    fold_no += 1

mean_loss = np.mean(loss_per_fold)
mean_loss

x_input = X1_scaled[-1].reshape((1, n_steps, n_features))
predictions = []
for i in range(7):
    yhat = model.predict(x_input, verbose=0)
    predictions.append(yhat[0,0])
    # Update the input for the next prediction
    x_input = array([x_input[0, 1:, 0].tolist() + [yhat[0,0]]])
    x_input = x_input.reshape((1, n_steps, n_features))

predictions = scaler.inverse_transform(np.array(predictions).reshape(-1, 1)).flatten()
print("Predictions for the next 7 periods:")
print(predictions)

```

```

X1_asli = np.array(df['Cattle'])
X1_ramal = np.array(predictions)

indices = np.arange(1968, 1968 + len(X1_asli) + len(X1_ramal))
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.plot(indices[:len(X1_asli)], X1_asli, label='Original Data')
plt.plot(indices[len(X1_asli):], X1_ramal, label='Forecast',
linestyle='dashed')
plt.xlabel('Year')
plt.ylabel('Value')
plt.title('Data Cattle with Forecast')
plt.legend()
plt.grid(True)
plt.show()

X2, y2 = split_sequence(x2, n_steps)
X2 = X2.reshape((X2.shape[0], X2.shape[1], n_features))
X2.shape

scaler = MinMaxScaler()

X2_scaled = scaler.fit_transform(X2.reshape(-1, X2.shape[-1])).reshape(X2.shape)
y2_scaled = scaler.fit_transform(y2.reshape(-1, y2.shape[-1])).reshape(y2.shape)

n_splits = 5
kf = KFold(n_splits=n_splits, shuffle=True)
fold_no = 1
loss_per_fold = []

for train_index, test_index in kf.split(X2_scaled):
    X_train, X_test = X2_scaled[train_index], X2_scaled[test_index]
    y_train, y_test = y2_scaled[train_index], y2_scaled[test_index]

    model = Sequential()
    model.add(Conv1D(filters=64, kernel_size=2, activation='relu',
input_shape=(n_steps, X2_scaled.shape[2])))
    model.add(MaxPooling1D(pool_size=2))
    model.add(Flatten())
    model.add(Dense(50, activation='relu'))
    model.add(Dense(1))
    model.compile(optimizer='adam', loss='mse')

    print(f"Training for fold {fold_no}...")
    model.fit(X_train, y_train, epochs=200, verbose=0)

    loss = model.evaluate(X_test, y_test, verbose=0)
    print(f"Fold {fold_no} loss: {loss}")

    loss_per_fold.append(loss)
    fold_no += 1

mean_loss = np.mean(loss_per_fold)

```

```

mean_loss

x_input = X2_scaled[-1].reshape((1, n_steps, n_features))
predictions = []
for i in range(7):
    yhat = model.predict(x_input, verbose=0)
    predictions.append(yhat[0,0])
    # Update the input for the next prediction
    x_input = array([x_input[0, 1:, 0].tolist() + [yhat[0,0]]])
    x_input = x_input.reshape((1, n_steps, n_features))

predictions = scaler.inverse_transform(np.array(predictions).reshape(-1,
1)).flatten()
print("Predictions for the next 7 periods:")
print(predictions)

X2_asli = np.array(df['Milk'])
X2_ramal = np.array(predictions)
indices = np.arange(1968, 1968 + len(X2_asli) + len(X2_ramal))
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.plot(indices[:len(X2_asli)], X2_asli, label='Original Data')
plt.plot(indices[len(X2_asli):], X2_ramal, label='Forecast',
linestyle='dashed')
plt.xlabel('Year')
plt.ylabel('Value')
plt.title('Data Milk with Forecast')
plt.legend()
plt.grid(True)
plt.show()

## LSTM
def split_sequence(sequence, n_steps):
    X, y = list(), list()
    for i in range(len(sequence)):
        # find the end of this pattern
        end_ix = i + n_steps
        # check if we are beyond the sequence
        if end_ix > len(sequence)-1:
            break
        # gather input and output parts of the pattern
        seq_x, seq_y = sequence[i:end_ix], sequence[end_ix]
        X.append(seq_x)
        y.append(seq_y)
    return array(X), array(y)

x1 = array(df['Cattle'])
x2 = array(df['Milk'])

x1 = x1.reshape((len(x1), 1))
x2 = x2.reshape((len(x2), 1))

n_steps = 5
n_features = 1

X1, y1 = split_sequence(x1, n_steps)

```



```

X1 = X1.reshape((X1.shape[0], X1.shape[1], n_features))
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

X1_scaled = scaler.fit_transform(X1.reshape(-1, X1.shape[-1])).reshape(X1.shape)
y1_scaled = scaler.fit_transform(y1.reshape(-1, y1.shape[-1])).reshape(y1.shape)
from sklearn.model_selection import KFold

n_splits = 5
kf = KFold(n_splits=n_splits, shuffle=True)

fold_no = 1
loss_per_fold = []

for train_index, test_index in kf.split(X1_scaled):
    X1_train, X1_test = X1_scaled[train_index], X1_scaled[test_index]
    y1_train, y1_test = y1_scaled[train_index], y1_scaled[test_index]

    model1 = Sequential()
    model1.add(LSTM(48, activation='relu', return_sequences=True,
input_shape=(n_steps, n_features)))
    model1.add(LSTM(24, activation='relu'))
    model1.add(Dense(1))
    model1.compile(optimizer='adam', loss='mse')

    print(f"Training for fold {fold_no}...")
    model1.fit(X1_train, y1_train, epochs=200, verbose=0)

    loss = model1.evaluate(X1_test, y1_test, verbose=0)
    print(f"Fold {fold_no} loss: {loss}")

    loss_per_fold.append(loss)
    fold_no += 1
mean_loss = np.mean(loss_per_fold)
mean_loss
model1.fit(X1_scaled, y1_scaled, epochs=200, verbose=0)

x1_input = X1_scaled[-1].reshape((1, n_steps, n_features))

predictions = []
for i in range(7):
    yhat1 = model1.predict(x1_input, verbose=0)
    predictions.append(yhat1[0,0])
    # Update the input for the next prediction
    x1_input = np.append(x1_input[0, 1:, 0], yhat1[0,0])
    x1_input = x1_input.reshape((1, n_steps, n_features))

predictions_inverse =
scaler.inverse_transform(np.array(predictions).reshape(-1, 1)).flatten()
print("Predictions for the next 7 periods:")
print(predictions_inverse)

```

```

X1_asli = np.array(df['Cattle'])
X1_ramal = np.array(predictions_inverse)

indices = np.arange(1968, 1968 + len(X1_asli) + len(X1_ramal))
plt.figure(figsize=(10, 6))
plt.plot(indices[:len(X1_asli)], X1_asli, label='Original Data')
plt.plot(indices[len(X1_asli):], X1_ramal, label='Forecast',
linestyle='dashed')
plt.xlabel('Year')
plt.ylabel('Value')
plt.title('Original Data with Forecast')
plt.legend()
plt.grid(True)
plt.show()

X2, y2 = split_sequence(x2, n_steps)
X2 = X2.reshape((X2.shape[0], X2.shape[1], n_features))
scaler = MinMaxScaler()

X2_scaled = scaler.fit_transform(X2.reshape(-1, X2.shape[-1])).reshape(X2.shape)
y2_scaled = scaler.fit_transform(y2.reshape(-1, y2.shape[-1])).reshape(y2.shape)
n_splits = 5
kf = KFold(n_splits=n_splits, shuffle=True)

fold_no = 1
loss_per_fold = []

for train_index, test_index in kf.split(X2_scaled):
    X2_train, X2_test = X2_scaled[train_index], X2_scaled[test_index]
    y2_train, y2_test = y2_scaled[train_index], y2_scaled[test_index]

    model2 = Sequential()
    model2.add(LSTM(48, activation='relu', return_sequences=True,
input_shape=(n_steps, n_features)))
    model2.add(LSTM(24, activation='relu'))
    model2.add(Dense(1))
    model2.compile(optimizer='adam', loss='mse')

    print(f"Training for fold {fold_no}...")
    model2.fit(X2_train, y2_train, epochs=200, verbose=0)

    loss = model2.evaluate(X2_test, y2_test, verbose=0)
    print(f"Fold {fold_no} loss: {loss}")

    loss_per_fold.append(loss)
    fold_no += 1
mean_loss = np.mean(loss_per_fold)
mean_loss
model2.fit(X2_scaled, y2_scaled, epochs=200, verbose=0)

x2_input = X2_scaled[-1].reshape((1, n_steps, n_features))

predictions2 = []

```

```

for i in range(7):
    yhat2 = model2.predict(x2_input, verbose=0)
    predictions2.append(yhat2[0,0])
    # Update the input for the next prediction
    x2_input = np.append(x2_input[0, 1:, 0], yhat2[0,0])
    x2_input = x2_input.reshape((1, n_steps, n_features))

predictions2_inverse =
scaler.inverse_transform(np.array(predictions2).reshape(-1,
1)).flatten()
print("Predictions for the next 7 periods:")
print(predictions2_inverse)

X2_asli = np.array(df['Milk'])
X2_ramal = np.array(predictions2_inverse)

indices = np.arange(1968, 1968 + len(X2_asli) + len(X2_ramal))
plt.figure(figsize=(10, 6))
plt.plot(indices[:len(X2_asli)], X2_asli, label='Original Data')
plt.plot(indices[len(X2_asli):], X2_ramal, label='Forecast',
linestyle='dashed')
plt.xlabel('Year')
plt.ylabel('Value')
plt.title('Original Data with Forecast')
plt.legend()
plt.grid(True)
plt.show()

## Bi-LSTM
### Prediksi Cattle Bi-LSTM
x1 = array(df['Cattle'])
x1 = x1.reshape((len(x1), 1))

def split_sequence(sequence, n_steps):
    X, y = list(), list()
    for i in range(len(sequence)):
        # find the end of this pattern
        end_ix = i + n_steps
        # check if we are beyond the sequence
        if end_ix > len(sequence)-1:
            break
        # gather input and output parts of the pattern
        seq_x, seq_y = sequence[i:end_ix], sequence[end_ix]
        X.append(seq_x)
        y.append(seq_y)
    return array(X), array(y)

n_steps = 5
X1, y1 = split_sequence(x1, n_steps)

n_features = 1
X1 = X1.reshape((X1.shape[0], X1.shape[1], n_features))

scaler = MinMaxScaler()

```

```

X1_scaled = scaler.fit_transform(X1.reshape(-1, X1.shape[-1])).reshape(X1.shape)
y1_scaled = scaler.fit_transform(y1.reshape(-1, y1.shape[-1])).reshape(y1.shape)

n_splits = 5
kf = KFold(n_splits=n_splits, shuffle=True)

fold_no = 1
loss_per_fold_cattle = []

for train_index, test_index in kf.split(X1_scaled):
    X1_train, X1_test = X1_scaled[train_index], X1_scaled[test_index]
    y1_train, y1_test = y1_scaled[train_index], y1_scaled[test_index]

    model = Sequential()
    model.add(Bidirectional(LSTM(48, activation='relu'),
input_shape=(n_steps, n_features)))
    model.add(Dense(1))
    model.compile(optimizer='adam', loss='mse')

    print(f"Training for fold {fold_no}...")
    model.fit(X1_train, y1_train, epochs=200, verbose=0)

    loss = model.evaluate(X1_test, y1_test, verbose=0)
    print(f"Fold {fold_no} loss: {loss}")

    loss_per_fold_cattle.append(loss)
    fold_no += 1

mean_loss_cattle= np.mean(loss_per_fold_cattle)
mean_loss_cattle

### Prediksi Milk Bi-LSTM

x2 = array(df['Milk'])
x2 = x2.reshape((len(x2), 1))

n_steps = 5
X2, y2 = split_sequence(x2, n_steps)
n_features = 1

X2 = X2.reshape((X2.shape[0], X2.shape[1], n_features))

from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

X2_scaled = scaler.fit_transform(X2.reshape(-1, X2.shape[-1])).reshape(X2.shape)
Y2_scaled = scaler.fit_transform(y2.reshape(-1, y2.shape[-1])).reshape(y2.shape)

from sklearn.model_selection import KFold

n_splits = 5
kf = KFold(n_splits=n_splits, shuffle=True)

```

```

fold_no = 1
loss_per_fold_milk = []

for train_index, test_index in kf.split(X1_scaled):
    X2_train, X2_test = X2_scaled[train_index], X2_scaled[test_index]
    y2_train, y2_test = y2_scaled[train_index], y2_scaled[test_index]

    model = Sequential()
    model.add(Bidirectional(LSTM(48, activation='relu'),
input_shape=(n_steps, n_features)))
    model.add(Dense(1))
    model.compile(optimizer='adam', loss='mse')

    print(f"Training for fold {fold_no}...")
    model.fit(X2_train, y2_train, epochs=200, verbose=0)

    loss = model.evaluate(X2_test, y2_test, verbose=0)
    print(f"Fold {fold_no} loss: {loss}")

    loss_per_fold_milk.append(loss)
    fold_no += 1

mean_loss_milk= np.mean(loss_per_fold_milk)
mean_loss_milk

```